

Tutorial – 6

Array, Linked List and Complexity

Q1. Convert a 2-D matrix into a linked list in the way shown below (each element points to the element below and the element to its right).

Input : 2D Matrix

```
1  2  3
4  5  6
7  8  9
```

Output :

```
1  -> 2  -> 3  -> NULL
|      |      |
v      v      v
4  -> 5  -> 6  -> NULL
|      |      |
v      v      v
7  -> 8  -> 9  -> NULL
|      |      |
v      v      v
NULL NULL NULL
```

Solution:

1. Assuming the matrix's dimension to be $m \times n$, create m separate linked lists, one list for each row.
2. The head node pointers of m linked lists are stored in an array.
3. Traverse through m lists. For every pair $(i, i+1)$ of lists, set the pointers of each node of the list i to the corresponding node of list $i+1$.

Detailed Solution: [Link](#)

Q2. Determine pairs in a doubly-linked list whose sum equals the provided value 'val' without consuming any extra space. The list is sorted and has only positive distinct entries. The expected complexities are $O(n)$ time and $O(1)$ space.

Solution:

1. Take two pointer variables P and Q .
2. Initialize the pointers. $P = \text{head}$ and $Q = \text{tail}$
3. If tail is not given, then linearly traverse the linked list and stop at the first node having value $\geq \text{val}$. We need not go all the way till the tail. Q points to such first node.
4. If $(P \rightarrow \text{data} + Q \rightarrow \text{data}) == \text{Val}$, then print the pair
5. If $(P \rightarrow \text{data} + Q \rightarrow \text{data}) < \text{Val}$, then $P = P \rightarrow \text{next}$

6. If $(P \rightarrow \text{data} + Q \rightarrow \text{data}) > \text{Val}$, then $Q = Q \rightarrow \text{prev}$
7. Repeat steps 4 to 7 till either the two pointers cross each other ($Q \rightarrow \text{next} == P$) or they become equal ($P == Q$).

Code: [Link](#)

Q3. Given a linked list of coordinates where adjacent points either form a vertical line or a horizontal line. Delete points from the linked list which are in the middle of a horizontal or vertical line. An example test case is shown below.

Input:

```
(0,10) -> (1,10) -> (5,10) -> (7,10)
                        |
                        (7,5) -> (20,5) -> (40,5)
```

Output: Linked List should be changed to following

```
(0,10) -> (7,10)
        |
        (7,5) -> (40,5)
```

The given linked list represents a horizontal line from (0,10) to (7, 10) followed by a vertical line from (7, 10) to (7, 5), followed by a horizontal line from (7, 5) to (40, 5).

Solution:

The idea is to keep track of the current node, next node, and next-next node. While the next node is the same as the next-next node, keep deleting the next node. In this complete procedure, we need to keep an eye on the shifting of pointers and checking for NULL values.

Detailed Solution: [link](#)

Q4. Given an array, rotate the array by one position in clockwise direction using $O(1)$ space. Compute the time complexity of your algorithm.

Example:

Input: $N = 5$, $A[] = \{1, 2, 3, 4, 5\}$ Output: 5 1 2 3 4

Solution:

Following are steps.

- 1) Store last element in a variable say x.
- 2) Shift all elements one position ahead.
- 3) Replace first element of array with x.

Assuming that each array operation (storing, shifting, and replacing) takes c time,

$$\text{Time taken (t)} = c + c(n-1) + c = 2c + c(n-1) = c(2+n-1) = c(n+1) = O(n)$$

Q5 Given an array, rotate the array by d positions in clockwise direction using $O(d)$ space. what is the time taken by your algorithm? How will your answer change if you were to do d rotations with only $O(1)$ auxiliary space?

Example:

Input: $N = 5, d = 3$ $A[] = \{1, 2, 3, 4, 5\}$ Output: 3 4 5 1 2

Solution:

- 1) Store last d elements in a variable array say arr.
- 2) Shift all other elements d positions ahead.
- 3) Replace first d elements of array with arr.

Assuming that each array operation (moving an element within an array or across two arrays) takes c time,

$$\begin{aligned}\text{Time taken} &= dc + c(n-d) + dc \\ &= 2dc + c(n-d) = c(2d+n-d) = c(n+d) = O(n+d)\end{aligned}$$

If we wanted to use only $O(1)$ space, do d rotations sequentially.

$$\text{Time taken} = dc(n+1) = O(nd)$$

Q6. Solve question 4 without using any auxiliary space. Compute the time complexity of your algorithm.

for($i=0; i < n-1; i++$)

Swap $A[i]$ with $A[n-1]$

Assuming each swap to take a constant time c

$$t = c(n-1) = O(n)$$

For brainstorming: How will you make d rotations without using any auxiliary space?

Q7. Find time and space complexity of the following code snippet.

```
int a = 0, b = 0;
for (i = 0; i < N; i++) {
    a = a + rand();
}
for (j = 0; j < M; j++) {
    b = b + rand();
}
```

Solution:

The first loop is $O(N)$ and the second loop is $O(M)$. Since we don't know which is bigger, we say this is $O(N + M)$. This can also be written as $O(\max(N, M))$.

Since there is no additional space being utilized, the space complexity is constant / $O(1)$

Q8. Find time complexity of the following code snippet.

```
int a[n];
for(int i = 0; i < n; i++)
    cin >> a[i]
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        if(i!=j && a[i]+a[j] == z)
            return true
    return false
```

Solution: Assuming that each of the operations in the computer takes approximately constant time, let it be c . The number of lines of code executed actually depends on the value of Z . During analyses of the algorithm, mostly the worst-case scenario is considered, i.e., when there is no pair of elements with sum equals Z . In the worst case,

- $N*c$ operations are required for input.

- The outer loop **i** loop runs **N** times.
- For each **i**, the inner loop **j** loop runs **N** times.

So total execution time is $N \cdot c + N \cdot N \cdot c + c$. Now ignore the lower order terms since the lower order terms are relatively insignificant for large input, therefore only the highest order term is taken (without constant) which is $N \cdot N$ in this case.

Q9. Find time complexity of the following code snippet.

```
count = 0
for (int i = N; i > 0; i /= 2)
for (int j = 0; j < i; j++)
count++;
```

Solution:

This is a tricky case. In the first look, it seems like the complexity is $O(N \cdot \log N)$. **N** for the **j**'s loop and $\log(N)$ for **i**'s loop. But it's wrong. Let's see why.

Think about how many times **count++** will run.

- When **i = N**, it will run **N** times.
- When **i = N / 2**, it will run **N / 2** times.
- When **i = N / 4**, it will run **N / 4** times.
- And so on.

The total number of times **count++** will run is $N + N/2 + N/4 + \dots + 1 = 2 \cdot N$. So the time complexity will be $O(N)$.

Q10. Find space complexity of the following code snippet.

```
int freq[n];
int a[n];
for(int i = 0; i < n; i++)
{
    cin >> a[i];
    freq[a[i]]++;
}
```

Solution:

Here two arrays of length N , and variable i are used in the algorithm so, the total space used is $N * c + N * c + 1 * c = 2N * c + c$, where c is a unit space taken. For many inputs, constant c is insignificant, and it can be said that the space complexity is $O(N)$.

There is also **auxiliary space**, which is different from space complexity. The main difference is where space complexity quantifies the total space used by the algorithm, auxiliary space quantifies the extra space that is used in the algorithm apart from the given input. In the above example, the auxiliary space is the space used by the `freq[]` array because that is not part of the given input. So total auxiliary space is $N * c + c$ which is $O(N)$ only.

Q11. Find time complexity of the following code snippet.

```
int fun(int n)
{
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j < n; j += i)
        {
            // Some O(1) task
        }
    }
}
```

Solution:

For $i = 1$, the inner loop is executed n times.

For $i = 2$, the inner loop is executed approximately $n/2$ times.

For $i = 3$, the inner loop is executed approximately $n/3$ times.

For $i = 4$, the inner loop is executed approximately $n/4$ times.

.....

.....

For $i = n$, the inner loop is executed approximately n/n times.

So the total time complexity of the above algorithm is $(n + n/2 + n/3 + \dots + n/n)$

Which becomes $n * (1/1 + 1/2 + 1/3 + \dots + 1/n)$

The important thing about series $(1/1 + 1/2 + 1/3 + \dots + 1/n)$ is, it is equal to $\Theta(\log n)$

Q12. What is the time complexity of below code?

```
void fun(int n)
{
    int j = 1, i = 0;
    while (i < n)
    {
        // Some O(1) task
        i = i + j;
        j++;
    }
}
```

Solution:

The loop variable 'i' is incremented by 1, 2, 3, 4, ... until i becomes greater than or equal to n.

The value of i is $x(x+1)/2$ after x iterations. So if loop runs x times, then $x(x+1)/2 < n$. Therefore time complexity can be written as $\Theta(\sqrt{n})$.